Adversarial Robustness Before and After Neural Network Compression

Nazia Afreen Md Atiqur Rahman

April 2026

Abstract

Neural network compression through pruning is widely used to reduce model size and computational cost while preserving clean accuracy. However, matching clean accuracy alone does not guarantee that a compressed model retains the same security behavior as its over-parameterized counterpart. In this project, we study how model compression affects two key security properties: robustness to test-time adversarial perturbations and vulnerability to train-time poisoning attacks. We evaluate dense and pruned versions of `VGG-16` and `ResNet-18` on `CIFAR-10` and `CIFAR-100` under matched clean accuracy settings using global unstructured magnitude-based pruning. For test-time security, we measure robustness under Fast Gradient Sign Method (FGSM) attacks and transfer attacks across multiple sparsity levels and perturbation strengths. For train-time security, we examine whether pruning changes the persistence of backdoor poisoning and label flipping attacks by comparing clean accuracy drop and attack success rate before and after compression.

Our results show that pruning does not reliably improve security, even when clean accuracy remains similar. Under test-time FGSM attacks, dense and pruned models usually show similar vulnerability, and transfer attacks remain highly effective between them. This suggests that pruning often preserves many of the same weak points rather than removing them. For train-time poisoning, pruning has more mixed effects. Low poison-rate backdoors are often strengthened after pruning even when clean accuracy changes little. Label flipping behaves less consistently and is not reliably removed by compression. Overall, these findings suggest that parameter redundancy does not consistently serve as a security buffer, and removing that redundancy through pruning does not reliably make models more secure.

1 Introduction

Modern deep neural networks are often highly over-parameterized, containing far more parameters than may be strictly necessary for strong predictive performance [4, 8]. This redundancy has made large models highly effective across

many tasks, but it has also encouraged widespread use of model compression techniques such as pruning. Pruning is used mainly to reduce deployment cost, memory footprint, and inference latency. In practice, pruning is typically evaluated based on efficiency gains and preservation of clean test accuracy [3]. However, clean accuracy alone does not fully capture whether a compressed model behaves similarly to its dense counterpart under security-relevant conditions.

Machine learning models face security threats at multiple stages of their lifecycle. During deployment, adversaries may craft carefully perturbed inputs that appear nearly unchanged to humans but cause incorrect predictions, as seen in test-time adversarial attacks such as FGSM [5]. During training, attackers may poison data through methods such as backdoor injection or label manipulation, causing hidden malicious behavior that may remain dormant until triggered [2, 6]. These threats raise an important question: **if over-parameterization provides implicit redundancy, does that redundancy also act as a form of fault tolerance against security attacks, and what happens when pruning removes it?**

This project studies adversarial robustness before and after model compression by examining whether pruning changes two distinct security properties under matched clean accuracy: *robustness to test-time adversarial perturbations* and *vulnerability to train-time poisoning*. Using `VGG-16` and `ResNet-18` [13, 9] models on `CIFAR-10` and `CIFAR-100` datasets [11], we compare dense and pruned models across multiple sparsity levels using global unstructured pruning [7]. For test-time security, we evaluate standard and transfer FGSM attacks. For train-time security, we measure the persistence of backdoor poisoning and label flipping attacks after pruning.

Our goal is not just to test whether pruning improves or weakens robustness, but to understand whether compression changes a model's security behavior even when clean accuracy stays similar. By studying both test-time and train-time threats together, we aim to better understand how compression affects machine learning security beyond standard accuracy measures.

1.1 Contributions

This project makes three primary contributions:

1. We evaluate whether model compression changes adversarial robustness under matched clean accuracy by comparing dense and pruned models against standard FGSM and transfer FGSM attacks across multiple sparsity levels.

2. We analyze whether pruning alters train-time attack persistence by studying how backdoor poisoning and label flipping behavior survive, weaken, or intensify after compression.

3. We provide a two-track security perspective that jointly studies test-time and training-time threats, offering a broader understanding of whether parameter redundancy serves as implicit security-related fault tolerance.

To support reproducibility, our project notebooks and implementation resources are publicly available on GitHub.[1]

2 Related Work

Research on neural network compression has traditionally focused on reducing model size and computational cost while preserving predictive performance. Early foundational work by Han et al. [8] introduced magnitude-based pruning as an effective way to remove less important weights while maintaining accuracy through retraining. This work demonstrated that large networks often contain substantial redundancy, motivating widespread interest in pruning as a deployment strategy. Since then, more pruning approaches have aimed to preserve clean accuracy [7, 14], but far fewer have examined whether compressed models also preserve broader security properties.

A growing body of work has shown that robustness and clean accuracy are not necessarily aligned. In the context of adversarial robustness, recent studies suggest that pruning can significantly alter how models respond to test-time attacks. Bair et al. [1] proposed adaptive sharpness-aware pruning, showing that pruning designed with robustness objectives can improve sparse model resilience. Their findings imply that standard accuracy-focused pruning may overlook security consequences. Similarly, Zhong et al. [15] observed that structured pruning can change adversarial behavior even when standard predictive accuracy remains competitive. Together, these studies suggest that compression may reshape gradient behavior, feature reliance, and decision boundaries in ways that matter for adversarial vulnerability.

Beyond test-time attacks, pruning has also been explored as a potential defense against train-time poisoning, especially backdoor attacks. Huang and Bu [10] proposed adversarial feature map pruning to suppress backdoor-related internal activations, while Li et al. [12] studied neuron pruning as a practical backdoor mitigation strategy. These works suggest that malicious behavior may sometimes be localized to specific subnetworks, allowing pruning to reduce attack success. However, they primarily treat pruning as a defense mechanism rather than asking whether standard compression itself naturally removes or preserves hidden attacks.

Our work differs from prior studies in two important ways. First, rather than focusing only on adversarial robustness or only on backdoor defense, we jointly evaluate both test-time and train-time security. Second, instead of designing specialized robustness-aware or defense-oriented pruning methods, we study standard global unstructured pruning under matched clean accuracy conditions to isolate how compression alone changes security behavior. This allows us to directly test whether the redundancy of over-parameterized models provides implicit fault tolerance and whether pruning disrupts that property.

[1] `https://github.com/NaziaAfreen015/CSC791-DLBA`

3 Background Knowledge

To understand how model compression may influence security, it is important to consider the two major stages where machine learning models can be attacked: deployment time and training time. In this project, we study both test-time adversarial attacks and train-time poisoning attacks to evaluate whether pruning changes model behavior under these security threats.

3.1 Test-time Adversarial Attacks

Test-time adversarial attacks aim to manipulate a model's prediction after training. Instead of changing the training data, the attacker adds carefully crafted perturbations to test input samples during deployment. These perturbations are often small enough to appear visually unchanged to humans, yet they can significantly alter model predictions by exploiting sensitivity to input gradients. In this project, we study two test-time adversarial attack strategies:

Fast Gradient Sign Method (FGSM): FGSM is a one-step white-box adversarial attack introduced by Goodfellow et al. [5]. It generates adversarial examples by perturbing an input in the direction that increases the model's loss:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y))$$

where x is the original input, y is the true label, θ represents model parameters, L is the loss function, and ϵ controls perturbation strength. Larger ϵ values usually produce stronger attacks but may make the perturbation more visible.

Transfer FGSM: Transfer FGSM evaluates whether adversarial examples generated on one model can fool another model. In this project, we craft adversarial examples on the dense model and test them on the pruned model, and also craft examples on the pruned model and test them on the dense model. This setting is closer to a black-box attack on the target model because the attacker does not use the target model's gradients directly. This is useful for understanding whether dense and pruned models share similar vulnerable decision boundaries. High transferability may indicate that both models rely on similar fragile features even after compression.

3.2 Train-time Poisoning Attacks

Train-time poisoning attacks target the learning process itself by modifying training data before or during model training. Instead of attacking deployed inputs, the attacker attempts to influence what the model learns [6]. In this project, we use two poisoning strategies:

Backdoor Poisoning: A subset of training samples is modified with a visible trigger pattern and relabeled to a target class. During training, the model learns to associate the trigger with the attacker's chosen label. As a result, the model may perform normally on clean data while consistently misclassifying triggered inputs at test time.

Label Flipping: Selected training samples are intentionally assigned incorrect labels without adding visible triggers. This introduces systematic label corruption and can bias decision boundaries, degrade accuracy, or create class-specific confusion.

4 Methodology

Our methodology compares the security behavior of dense and compressed neural networks while keeping clean accuracy as similar as possible. The core goal is to isolate the effect of pruning itself rather than differences caused simply by standard predictive performance changes. To do this, we organize the project into two complementary experimental tracks: (1) test-time adversarial robustness and (2) train-time poisoning vulnerability.

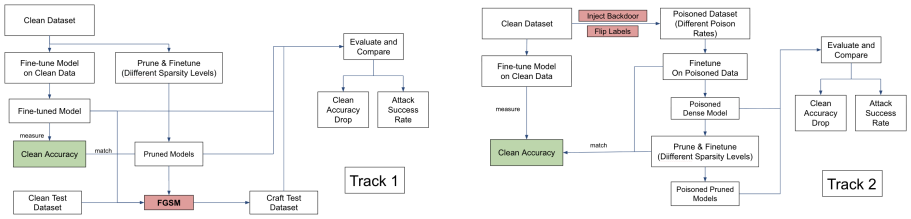
4.1 Experimental Setup

We use `CIFAR-10` and `CIFAR-100` as our primary datasets because they provide two different levels of classification complexity while remaining practical for repeated pruning and adversarial experiments. `CIFAR-10` contains 10 object classes, while `CIFAR-100` provides a more challenging 100-class setting [11]. For both datasets, we split the original training set into 90% training and 10% validation, reserve the official test set for final evaluation, and use a fixed random seed for reproducibility.

Our experiments use two ImageNet-pretrained architectures: `VGG-16` and `ResNet-18`. For each model, the final classifier layer is replaced to match the target dataset's class count. To align CIFAR inputs with pretrained ImageNet architectures, images are resized from 32×32 to 224×224. During training, we apply standard augmentation through random crop and horizontal flip, while validation and test data use normalization only.

For each architecture-dataset pair, we first fine-tune a dense baseline model to establish clean performance before applying any compression. `ResNet-18` uses a staged three-phase fine-tuning process, while `VGG-16` uses a two-stage process, allowing stable transfer from ImageNet-pretrained weights to CIFAR classification. Across all experiments, we use SGD with momentum, CrossEntropyLoss, and a StepLR scheduler, selecting the best checkpoint based on validation accuracy.

After establishing the dense baseline, we apply global unstructured magnitude based pruning following Han et al. [8]. We evaluate sparsity levels of 20%, 40%, 60%, and 80%. For each sparsity level, we perform one-shot pruning



(a) Track 1: Test-time adversarial robustness

(b) Track 2: Train-time poisoning vulnerability

Figure 1: Overall experimental design for both security tracks. Track 1 begins with dense baseline training, pruning at multiple sparsity levels, and FGSM-based evaluation on clean and adversarial test data to compare standard and transfer robustness between dense and pruned models. Track 2 begins by generating poisoned datasets through either backdoor injection or label flipping, then compares poisoned dense and poisoned pruned models under matched clean accuracy. Across both tracks, clean accuracy is first measured and matched before security-specific metrics are compared, ensuring that observed differences primarily reflect the effect of pruning rather than baseline performance loss.

followed by retraining to recover clean accuracy as closely as possible to the dense baseline. This matched clean accuracy design is critical because it allows security differences to be interpreted as changes caused by compression rather than simple predictive degradation.

4.2 Experimental Tracks

Track 1 evaluates test-time adversarial robustness after deployment. For this track, we primarily study FGSM attacks across multiple perturbation strengths, $\epsilon \in \{2/255,\ 4/255,\ 8/255,\ 16/255\}$. For each dense and pruned model, we first measure clean performance, then generate adversarial examples from correctly classified samples, and finally evaluate adversarial robustness. In addition to standard FGSM, we also evaluate transfer FGSM, where adversarial examples crafted on one model are tested on another (dense-to-pruned and pruned-to-dense). This helps determine whether pruning changes transferable vulnerabilities or whether dense and pruned models continue to rely on similar fragile features.

Track 2 evaluates train-time poisoning vulnerability by studying whether malicious behavior survives compression. We examine two poisoning strategies: backdoor poisoning and label flipping, each tested across poison rates of 1%, 5%, and 10%. For backdoor poisoning, we inject a fixed 5×5 white square trigger into selected training samples and relabel them to a target class before training. For label flipping, we corrupt selected labels directly, using selected-class flips for `CIFAR-10` and superclass-aware flips for `CIFAR-100`. In both cases, poisoned dense models are later pruned and retrained to determine whether pruning

weakens, preserves, or amplifies poisoned behavior.

4.3 Evaluation Metrics

For Track 1, we evaluate adversarial robustness using adversarial accuracy drop and attack success rate (ASR). Adversarial accuracy drop measures the decrease in accuracy after applying FGSM perturbations:

$$\text{Adversarial Accuracy Drop} = \text{Clean Accuracy} - \text{Adversarial Accuracy}$$

Attack success rate measures the fraction of initially correct samples that become misclassified under attack:

$$ASR = \frac{\#(\text{initially correct but incorrect under attack})}{\#(\text{initially correct})}$$

For transfer FGSM, we report the same metrics in both directions: dense-to-pruned and pruned-to-dense. Transfer ASR is computed using samples that the target model classifies correctly on clean inputs, and then measuring how often those same inputs become incorrect after applying adversarial examples crafted on the source model:

$$\text{Transfer ASR} = \frac{\#(\text{target clean correct and target adversarial wrong})}{\#(\text{target clean correct})}$$

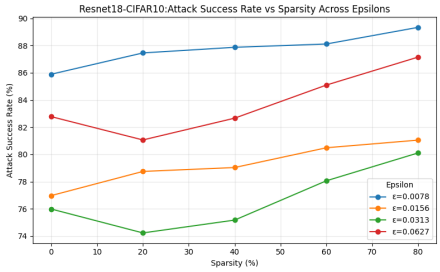
Together, these metrics show both the direct robustness of each model and the extent to which adversarial vulnerabilities transfer between dense and pruned models.

For Track 2, we use Clean Accuracy Drop (CAD) and Attack Success Rate (ASR). CAD measures the difference between dense baseline clean accuracy and the corresponding poisoned or pruned model. For backdoor poisoning, ASR measures the percentage of triggered test inputs that are predicted as the attacker's target class when the trigger is applied across the test set. More broadly, this reflects how effectively the poisoning objective remains active after pruning.

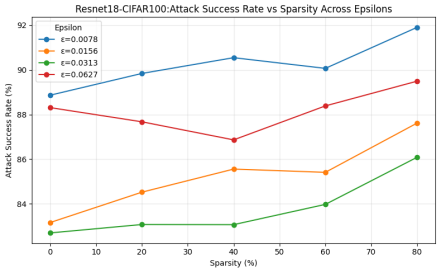
Together, these metrics allow us to evaluate whether compression preserves, weakens, or amplifies security-relevant behavior across both deployment-time and training-time threat settings.

5 Experimental Results

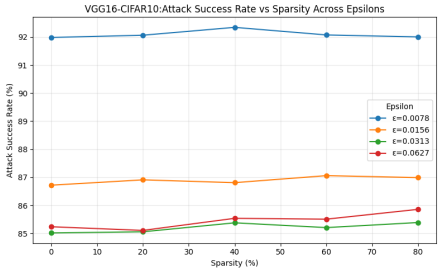
We present results for both experimental tracks: test-time adversarial robustness and train-time poisoning vulnerability. Across all experiments, we compare dense baselines with globally pruned models at 20%, 40%, 60%, and 80% sparsity after retraining.



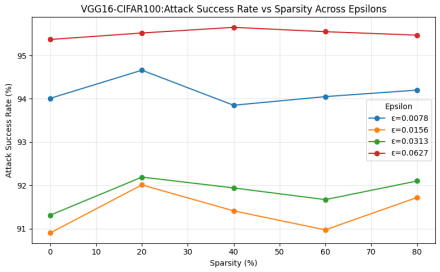
(a) `ResNet-18` on `CIFAR-10`



(b) `ResNet-18` on `CIFAR-100`



(c) `VGG-16` on `CIFAR-10`



(d) `VGG-16` on `CIFAR-100`

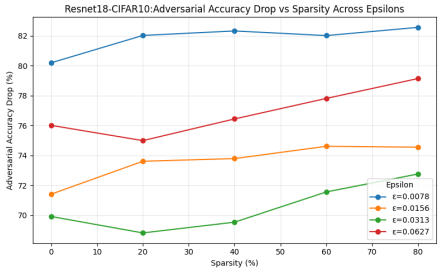
Figure 2: FGSM attack success rate across poison rates and pruning levels for `ResNet-18` and `VGG-16` on `CIFAR-10` and `CIFAR-100`. The results show that attack success rate (ASR) remains remarkably similar or even increases between dense and pruned models across sparsity levels.

5.1 Track 1: Test-time Adversarial Robustness

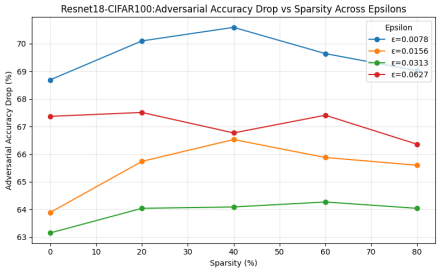
5.1.1 Standard FGSM Attack

We first evaluate standard white-box FGSM attacks on each model independently across multiple perturbation strengths. Figure 2 reports FGSM attack success rate (ASR), while Figure 3 reports adversarial accuracy drop across architectures, datasets, epsilons, and pruning levels.

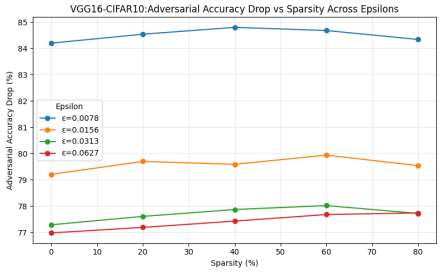
Across all four settings, FGSM attack success rate remains high across sparsity levels, showing that pruning does not consistently reduce adversarial vulnerability. For `VGG-16`, ASR changes only slightly as sparsity increases, with `CIFAR-10` staying around the mid-80% to low-90% range and `CIFAR-100` staying above 90% for all tested epsilons. In contrast, `ResNet-18` shows a clearer upward trend with sparsity, especially at higher pruning levels, where ASR generally increases as sparsity reaches 60% and 80%. This suggests that pruning tends to preserve FGSM vulnerability in `VGG-16`, while for `ResNet-18` higher sparsity may further weaken robustness. Overall, the models remain highly susceptible to FGSM attacks even after pruning, and increased sparsity does not provide reliable adversarial protection.



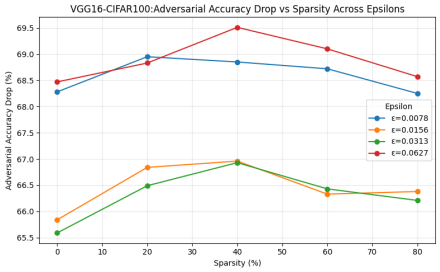
(a) `ResNet-18` on `CIFAR-10`



(b) `ResNet-18` on `CIFAR-100`



(c) `VGG-16` on `CIFAR-10`



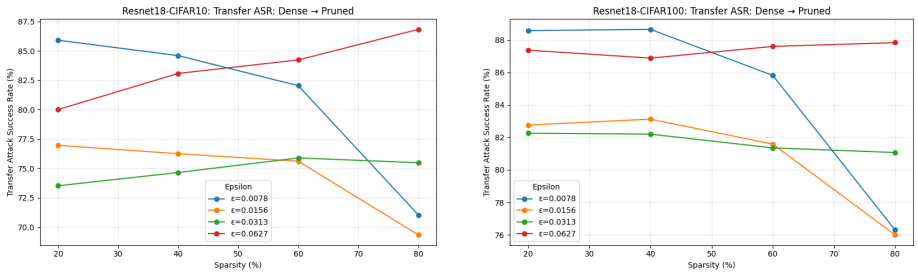
(d) `VGG-16` on `CIFAR-100`

Figure 3: FGSM adversarial accuracy drop across perturbation strengths and pruning levels for `ResNet-18` and `VGG-16` on `CIFAR-10` and `CIFAR-100`. The results show that adversarial accuracy drop remains substantial after pruning, with several settings showing similar or greater degradation as sparsity increases.

Across all four settings, adversarial accuracy drop remains large across sparsity levels, indicating that FGSM causes substantial performance degradation even after pruning. For `VGG-16:CIFAR-10`, the drop is relatively stable across sparsity, with only small fluctuations as pruning increases. `VGG-16:CIFAR-100` shows a mild increase up to around 40% sparsity, followed by a slight decrease at higher sparsity levels. For `ResNet-18`, the effect of pruning is more pronounced: on `CIFAR-10`, the accuracy drop generally increases with sparsity, especially at 60% and 80%, while on `CIFAR-100` the drop peaks around 40% sparsity and then decreases slightly. Overall, adversarial accuracy drop remains high across sparsity levels, and in several cases, especially for `ResNet-18`, higher sparsity is associated with equal or greater degradation under FGSM.

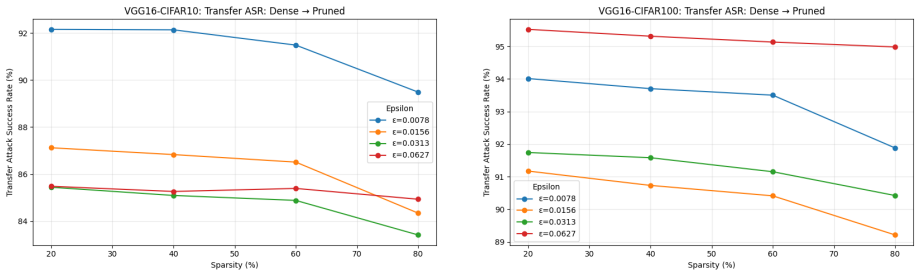
5.1.2 Transfer FGSM Attack

Transfer attacks reveal whether adversarial examples generated on one model remain effective against another. Figure 4 reports FGSM trasnfer attack success rate (ASR) from dense model to pruned model, while Figure 5 reports reports FGSM trasnfer attack success rate (ASR) from pruned model to dense model across dataset, architecture, epsilons, and sparsity levels.



(a) `ResNet-18` on `CIFAR-10`

(b) `ResNet-18` on `CIFAR-100`



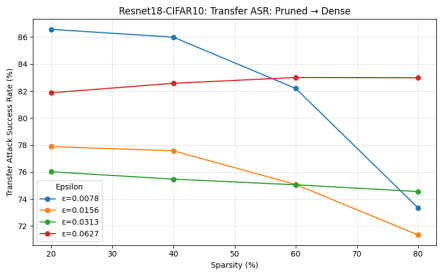
(c) `VGG-16` on `CIFAR-10`

(d) `VGG-16` on `CIFAR-100`

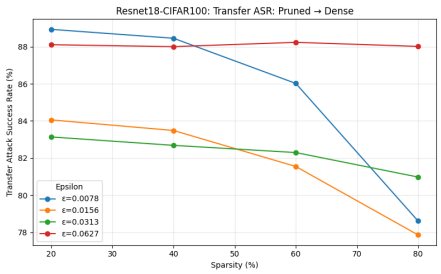
Figure 4: Dense → Pruned: Transfer ASR for FGSM examples generated on dense models and evaluated on pruned models across sparsity levels and perturbation strengths. The results show that pruning does not fully block dense-model adversarial examples; many attacks remain effective after compression, especially at moderate sparsity levels.

Across `VGG-16` experiments on both `CIFAR-10` and `CIFAR-100`, transfer ASR between dense and pruned models remained consistently high in both directions, dense-to-pruned and pruned-to-dense. This suggests that compression preserves many of the same vulnerable feature directions. For `ResNet-18`, transfer ASR is also high overall, but the trend is less stable. At moderate sparsity, transferability was often nearly unchanged. At higher sparsity, especially 80%, some lower-epsilon attacks transfer less effectively, while larger perturbations often remain strong. However, in both cases transfer success generally remained substantial. These findings are important because they suggest that compression does not necessarily create a fundamentally new security profile. Instead, pruned models often inherit similar weaknesses from their dense predecessors.

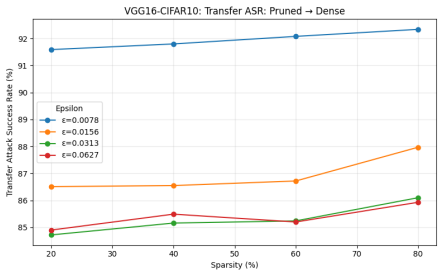
Overall, transfer FGSM results show that pruning may slightly change adversarial behavior at extreme sparsity, but it does not create a clearly safer model. Dense and pruned models still share many similar vulnerabilities.



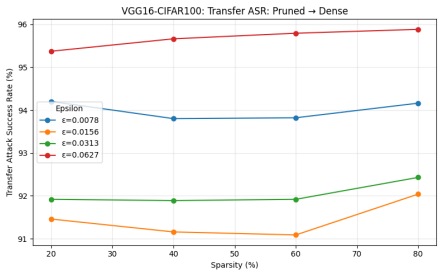
(a) `ResNet-18` on `CIFAR-10`



(b) `ResNet-18` on `CIFAR-100`



(c) `VGG-16` on `CIFAR-10`



(d) `VGG-16` on `CIFAR-100`

Figure 5: Pruned → Dense: Transfer FGSM attack success rate from pruned models to dense models across perturbation strengths and pruning levels for `ResNet-18` and `VGG-16` on `CIFAR-10` and `CIFAR-100`. The results show that adversarial examples crafted on pruned models also transfer strongly back to dense models, suggesting that dense and pruned models share many fragile feature directions.

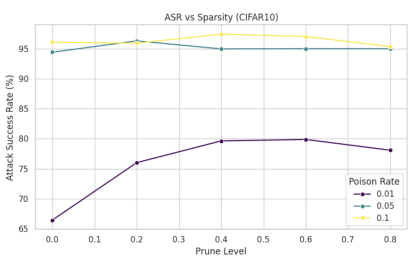
5.2 Track 2: Train-time Poisoning Vulnerability

5.2.1 Backdoor Poisoning

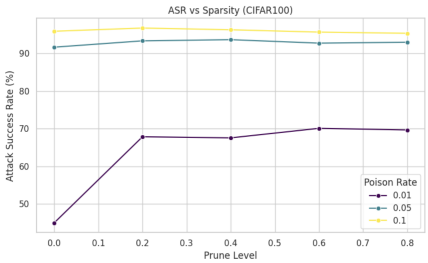
Figure 6 reports backdoor attack success rate (ASR), while Figure 7 reports clean accuracy drop (CAD) across architectures, datasets, poison rates, and pruning levels.

A consistent pattern across all settings is that low poison-rate backdoors, particularly at 1%, become more effective after pruning. For both `ResNet-18` and `VGG-16`, pruning substantially increases ASR from the poisoned dense baseline, with the largest jumps typically occurring between 20% and 40% sparsity. This suggests that standard pruning does not reliably remove subtle backdoor behavior and may instead strengthen it when the backdoor is initially weakly embedded. In contrast, at higher poison rates (5% and 10%), dense poisoned models already achieve very high ASR, so pruning produces smaller relative changes because the backdoor is already strongly learned.

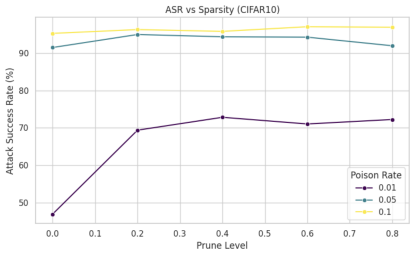
Clean accuracy shows a different trend. As shown in Figure 7, moderate



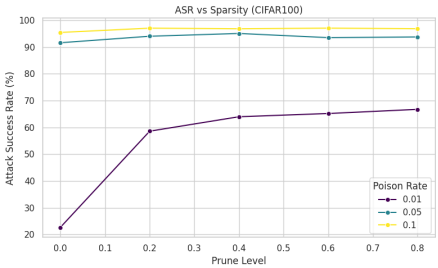
(a) ResNet-18 on CIFAR-10



(b) ResNet-18 on CIFAR-100



(c) VGG-16 on CIFAR-10



(d) VGG-16 on CIFAR-100

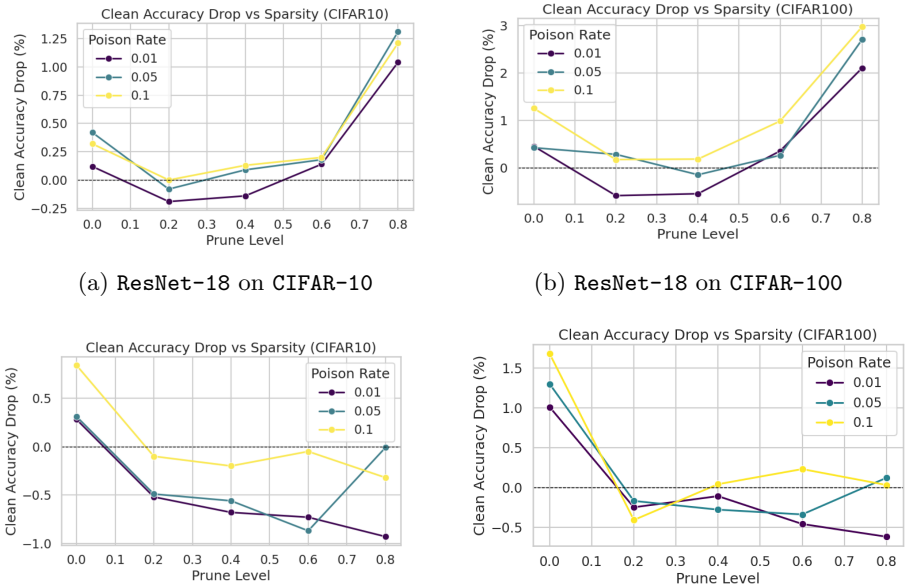
Figure 6: Backdoor attack success rate across poison rates and pruning levels for ResNet-18 and VGG-16 on CIFAR-10 and CIFAR-100. ASR measures the percentage of triggered test inputs classified as the target class. The results show that pruning often increases ASR at low poison rates, while its effect is smaller when the dense poisoned model already has high ASR.

pruning often preserves or only slightly changes clean performance, and in several VGG-16 settings even reduces clean accuracy drop relative to the poisoned dense baseline. Larger sparsity levels, particularly 80%, are more likely to increase clean accuracy loss, especially for ResNet-18. Taken together, these results show that pruning can maintain near-baseline clean accuracy while simultaneously increasing backdoor effectiveness, particularly at low poison rates. Overall, standard magnitude-based pruning does not reliably eliminate backdoors and, in several cases, may preserve or amplify malicious behavior despite limited impact on standard predictive performance.

5.2.2 Label Flipping

Label flipping differs from backdoor poisoning because no trigger is added to the input. Instead, the attack corrupts the training process by changing labels for selected samples.

For CIFAR-10, labels are flipped between the cat and dog classes, so we report both full clean accuracy drop and selected-class accuracy drop for those affected classes (Figure 8). Across both ResNet-18 and VGG-16, pruning often reduced



(a) `ResNet-18` on `CIFAR-10`

(b) `ResNet-18` on `CIFAR-100`

(c) `VGG-16` on `CIFAR-10`

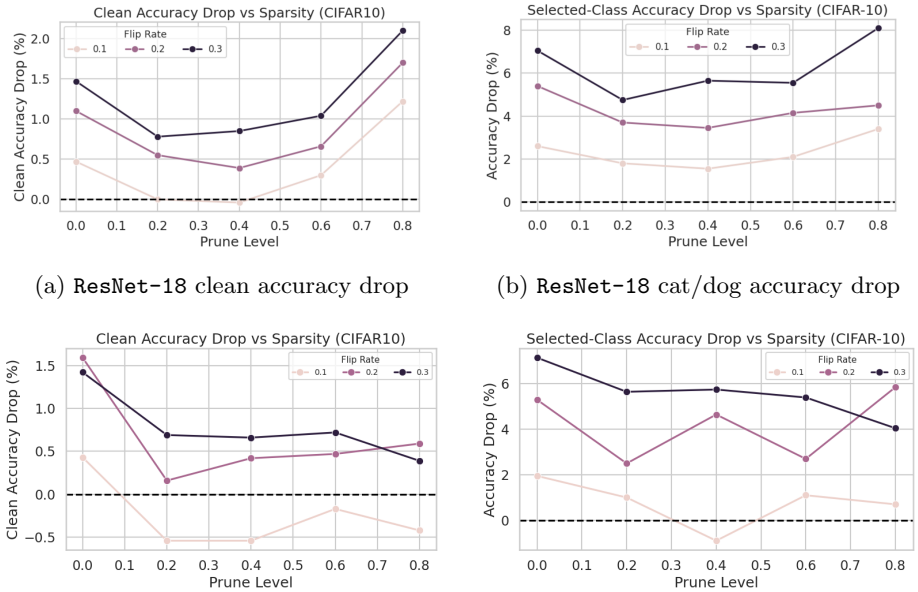
(d) `VGG-16` on `CIFAR-100`

Figure 7: Clean accuracy drop for backdoor-poisoned models across poison rates and pruning levels. CAD compares clean test accuracy against the dense clean baseline. These results help separate backdoor persistence from ordinary clean performance changes, showing that pruning can maintain or recover clean accuracy even when the backdoor remains active.

overall clean accuracy drop at moderate sparsity levels, particularly between 20% and 40%, compared to the unpruned poisoned baseline. However, selected-class accuracy drop remained consistently larger than overall clean accuracy drop, showing that while pruning may partially recover general performance, class-specific corruption often persists more strongly. At higher flip rates, especially 30%, both global and selected-class degradation remained substantial, with aggressive pruning sometimes increasing harm further.

For `CIFAR-100`, we use superclass-aware label flipping, where labels are flipped to another class within the same superclass rather than arbitrarily across all classes (Figure 9). This produces semantically related corruption and allows us to compare full clean accuracy drop with superclass accuracy drop. Similar to `CIFAR-10`, moderate pruning often reduced some clean accuracy degradation, but superclass-level damage generally remained lower than full clean accuracy drop, suggesting that broader semantic structure is often preserved even when fine-grained labels are corrupted. However, at high flip rates and extreme sparsity, both metrics worsened, particularly for `ResNet-18`.

Overall, label flipping produced more distributed and less predictable behavior than backdoor poisoning. Moderate pruning sometimes partially reduced the



(a) `ResNet-18` clean accuracy drop

(b) `ResNet-18` cat/dog accuracy drop

(c) `VGG-16` clean accuracy drop

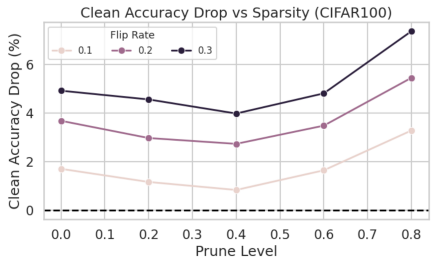
(d) `VGG-16` cat/dog accuracy drop

Figure 8: Label flipping results on `CIFAR-10`. The attack flips labels between the cat and dog classes, so we report both full clean accuracy drop and selected-class accuracy drop for those affected classes. This separates the global effect of poisoning from its class-specific impact.

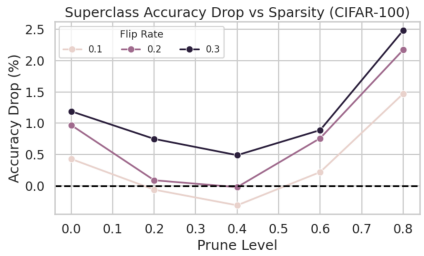
impact of corrupted supervision, especially at lower sparsity levels, but these improvements were inconsistent across architectures, datasets, and flip rates. Standard pruning therefore cannot be treated as a reliable defense against label flipping, even when some clean accuracy recovery is observed.

6 Conclusion

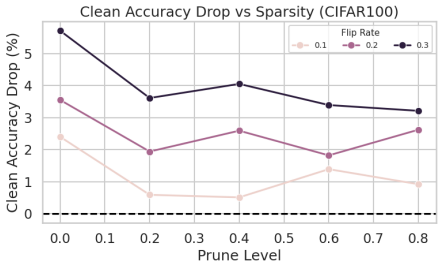
Model compression can preserve clean accuracy while still changing security behavior. Across both test-time and train-time settings, our results show that standard pruning does not consistently improve security. Under test-time adversarial attacks, pruning often preserves similar vulnerability and transferability. Under train-time poisoning attacks, its effects are more mixed. In some cases, pruning can preserve or even strengthen harmful behavior. Overall, these findings suggest that parameter redundancy does not consistently act as a security buffer. Removing that redundancy through pruning can change security behavior, but not always in ways that improve robustness.



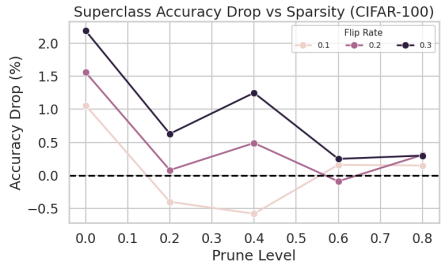
(a) `ResNet-18` clean accuracy drop



(b) `ResNet-18` superclass accuracy drop



(c) `VGG-16` clean accuracy drop



(d) `VGG-16` superclass accuracy drop

Figure 9: Label flipping results on `CIFAR-100`. Labels are flipped within the same superclass, so the attack corrupts fine-grained labels while keeping them semantically related. We report clean accuracy drop and superclass accuracy drop to show both the overall impact and the effect on broader category-level behavior.

Acknowledgments

We would like to thank our course instructor, Professor Jung-Eun Kim, for her guidance and for approving our project proposal, which helped shape the direction of this work. We also acknowledge the collaborative efforts of both authors in designing, implementing, and analyzing the two complementary security tracks explored in this project. Finally, we used AI-based language assistance tools solely to help refine writing clarity, grammar, and presentation style. All experimental design, implementation, analysis, and core technical content were developed by the authors.

References

[1] A. Bair, H. Yin, M. Shen, P. Molchanov, and J. Alvarez. Adaptive sharpness-aware pruning for robust sparse networks. *arXiv preprint arXiv:2306.14306*, 2023.

[2] B. Biggio, B. Nelson, and P. Laskov. Poisoning attacks against support vector machines. *arXiv preprint arXiv:1206.6389*, 2012.

[3] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2:129–146, 2020.

[4] M. Denil, B. Shakibi, L. Dinh, M. Ranzato, and N. De Freitas. Predicting parameters in deep learning. *Advances in neural information processing systems*, 26, 2013.

[5] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[6] T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain. arxiv 2017. *arXiv preprint arXiv:1708.06733*, 2017.

[7] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.

[8] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

[9] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[10] D. Huang and Q. Bu. Adversarial feature map pruning for backdoor. *arXiv preprint arXiv:2307.11565*, 2023.

[11] A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images.(2009), 2009.

[12] N. Li, H. Jiang, and P. Yi. Magnitude-based neuron pruning for backdoor defens. *arXiv preprint arXiv:2405.17750*, 2024.

[13] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[14] T. Suzuki, H. Abe, T. Murata, S. Horiuchi, K. Ito, T. Wachi, S. Hirai, M. Yukishima, and T. Nishimura. Spectral pruning: Compressing deep neural networks via spectral analysis and its generalization error. *arXiv preprint arXiv:1808.08558*, 2018.

[15] S. H. Zhong, Z. You, J. Zhang, S. Zhao, Z. LeClaire, Z. Liu, D. Zha, V. Chaudhary, S. Xu, and X. Hu. One less reason for filter pruning: Gaining free adversarial robustness with structured grouped kernel pruning. *Advances in neural information processing systems*, 36:62032–62061, 2023.

7 Member Contributions

This project was divided into two complementary research tracks:

- **Nazia Afreen**: Led Track 1, which focused on test-time adversarial robustness. This included dense and pruned model training for clean baselines, FGSM and transfer FGSM implementation, robustness evaluation across sparsity and perturbation levels, and comparative analysis of adversarial vulnerability before and after compression.
- **Md Atiqur Rahman**: Led Track 2, which focused on train-time adversarial attacks. This included poisoned dataset generation through backdoor poisoning and label flipping, poisoned model fine-tuning, pruning of compromised models, and analysis of attack success rate and clean accuracy changes across poison rates and sparsity levels.

While each member led a primary track, both collaborated on overall project design, experimental framing, interpretation of findings, and final report preparation.